

Project Report

Group 5

Stage 3

Ghala Alghamdi S23108212 – Hiba Amanulla S23108227 – Natali Soukh S23108314

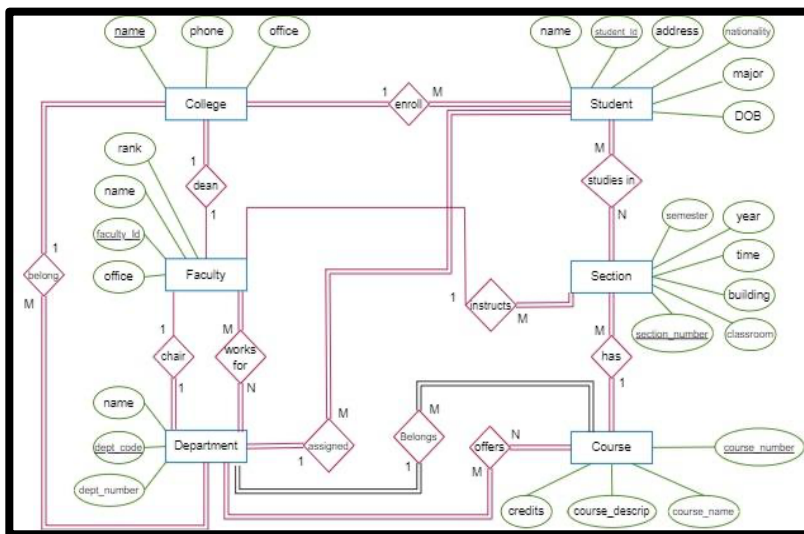
Introduction/

- Our application, *Student Records*, is a database system designed to keep track of all the students and faculty members' data, including colleges they are involved in and departments they are related to. It also organizes all the courses that are offered and its sections.
- We chose this application for its importance in a university's system and its complexity in managing all academic information. All the needed information should be kept neat and organized for easier access.
- The key components of our project include database implementation, frontend functionality, and advanced features aimed at providing a platform for managing a university's operations efficiently.

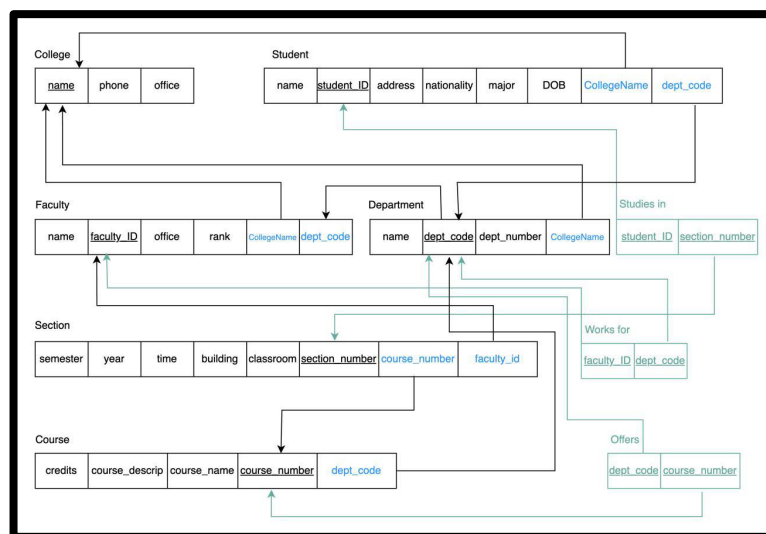
Database Details/

- In designing our database, we followed principles of entity-relationship (ER diagram) modeling to visualize the relationships between different entities.
- Our ER model and relational model:

ER Diagram



Relational Schema



- The tables included in our database involve multiple interconnected tables (offers, studies_in, works_for), involving already existing entities and attributes.

- Our tables:

College table

```
[MariaDB [university]> DESC College;
```

Field	Type	Null	Key	Default	Extra
CollegeName	varchar(255)	NO	PRI	NULL	
Phone	varchar(20)	YES		NULL	
Office	varchar(255)	YES		NULL	

```
3 rows in set (0.013 sec)
```

Course table

```
[MariaDB [university]> DESC Course;
```

Field	Type	Null	Key	Default	Extra
Course_number	varchar(20)	NO	PRI	NULL	
Course_Name	varchar(255)	YES		NULL	
Course_Descrip	text	YES		NULL	
Credits	int(11)	YES		NULL	
DepartmentCode	int(11)	YES	MUL	NULL	

```
5 rows in set (0.005 sec)
```

Student table

```
[MariaDB [university]> DESC Student;
```

Field	Type	Null	Key	Default	Extra
StudentID	int(11)	NO	PRI	NULL	
Name	varchar(255)	YES		NULL	
Address	varchar(255)	YES		NULL	
Nationality	varchar(50)	YES		NULL	
DateOfBirth	date	YES		NULL	
Major	varchar(100)	YES		NULL	
DepartmentCode	int(11)	YES	MUL	NULL	
CollegeName	varchar(255)	YES	MUL	NULL	

```
8 rows in set (0.004 sec)
```

```
[MariaDB [university]> SHOW TABLES;
```

Tables_in_university
College
Course
Department
Faculty
Offers
Section
Student
Studies_in
Works_for

```
9 rows in set (0.001 sec)
```

Department table

```
[MariaDB [university]> DESC Department;
```

Field	Type	Null	Key	Default	Extra
DepartmentCode	int(11)	NO	PRI	NULL	
DepartmentName	varchar(255)	NO		NULL	
DepartmentNumber	varchar(20)	YES		NULL	
CollegeName	varchar(255)	YES	MUL	NULL	

```
4 rows in set (0.005 sec)
```

Faculty table

```
[MariaDB [university]> DESC Faculty;
```

Field	Type	Null	Key	Default	Extra
FacultyID	int(11)	NO	PRI	NULL	
Name	varchar(255)	YES		NULL	
OfficeNumber	varchar(20)	YES		NULL	
PhoneNumber	varchar(20)	YES		NULL	
Rank	varchar(50)	YES		NULL	
DepartmentCode	int(11)	YES	MUL	NULL	
CollegeName	varchar(255)	YES	MUL	NULL	

```
7 rows in set (0.005 sec)
```

Section table

```
[MariaDB [university]> DESC Section;
```

Field	Type	Null	Key	Default	Extra
Section_number	int(11)	NO	PRI	NULL	
Semester	varchar(10)	YES		NULL	
Year	int(11)	YES		NULL	
Timing	varchar(50)	YES		NULL	
Classroom	varchar(20)	YES		NULL	
Building	varchar(50)	YES		NULL	
FacultyID	int(11)	YES	MUL	NULL	
Course_number	varchar(20)	YES	MUL	NULL	

```
8 rows in set (0.004 sec)
```

Offers interconnected table:
Course and department

```
MariaDB [university]> DESC Offers;
```

Field	Type	Null	Key	Default	Extra
Course_number	varchar(20)	NO	PRI	NULL	
DepartmentCode	int(11)	NO	PRI	NULL	

2 rows in set (0.003 sec)

Studies_in interconnected table:
Student and Section

```
MariaDB [university]> DESC Studies_in;
```

Field	Type	Null	Key	Default	Extra
StudentID	int(11)	NO	PRI	NULL	
Section_number	int(11)	NO	PRI	NULL	

2 rows in set (0.002 sec)

Works_for interconnected table:
Faculty and Department

```
MariaDB [university]> DESC Works_for;
```

Field	Type	Null	Key	Default	Extra
FacultyID	int(11)	NO	PRI	NULL	
DepartmentCode	int(11)	NO	PRI	NULL	

2 rows in set (0.005 sec)

- We ensured that our database is in 3NF to minimize redundancy.
- Various constraints were implemented to maintain data consistency.

Functionality Details/

- The Student Records database offers both basic and advanced functionalities to different user roles within the university.
- Basic functions include tasks such as: course enrollment, grade assignment, and student information retrieval.
- Advanced functions include features such as: a personalized teaching schedule for faculty members, dynamic course search functionality, and real-time updates on course availability.
- These functionalities are implemented using a combination of server-side and client-side scripting languages, with SQL queries for database manipulation.

Implementation Details/

- For the implementation of our project, we utilized XAMP, MySQL, and Python from VS Code, chosen for our project requirements.
- The backend database is implemented using MySQL, which we ran using XAMP. It provides a robust storage solution for our program data.
- We used Python and employed it to create the frontend interface.
- The interaction between the frontend interface and the backend database is facilitated through MySQL Connector between the MySQL database and our Python code, allowing for seamless data exchange without interruptions.
 - Program link from GitHub: <https://github.com/hibanausheen144/Student-Records>
 - Database tables:

College table contents

```
MariaDB [university]> SELECT * FROM College;
```

CollegeName	Phone	Office
Business School	(777) 999-8888	Business Building, Lobby
City College	(987) 654-3210	Main Building, Room 201
Liberal Arts College	(888) 222-3333	Arts Building, Room 301
Tech Institute	(555) 123-4567	Engineering Building, Dean's Office
University of Science	(123) 456-7890	Science Building, Room 101

5 rows in set (0.014 sec)

Department table

```
MariaDB [university]> SELECT * FROM Department;
```

DepartmentCode	DepartmentName	DepartmentNumber	CollegeName
101	Computer Science	CS101	University of Science
201	Mathematics	MATH201	City College
301	Electrical Engineering	EE301	Tech Institute
401	English Literature	ENGL401	Liberal Arts College
501	Business Administration	BUS501	Business School

5 rows in set (0.002 sec)

Course table contents

```
MariaDB [university]> SELECT * FROM Course;
```

Course_number	Course_Name	Course_Descrip	Credits	DepartmentCode
BUS501	Principles of Marketing	Introduction to marketing principles and strategies.	3	501
CS101	Introduction to Computer Science	Fundamental concepts of computer science.	3	101
EE301	Introduction to Electrical Engineering	Fundamental concepts of electrical engineering.	3	301
ENGL401	Shakespearean Literature	Study of Shakespeare's plays and sonnets.	3	401
MATH201	Calculus I	Fundamental concepts of calculus.	4	201

5 rows in set (0.003 sec)

Faculty table contents

```
MariaDB [university]> SELECT * FROM Faculty;
```

FacultyID	Name	OfficeNumber	PhoneNumber	Rank	DepartmentCode	CollegeName
1001	John Smith	S101	(123) 456-7898	Professor	101	University of Science
1002	Emily Johnson	M201	(987) 654-3210	Associate Professor	201	City College
1003	David Lee	E301	(555) 123-4567	Assistant Professor	301	Tech Institute
1004	Alice Brown	A401	(888) 222-3333	Professor	401	Liberal Arts College
1005	Michael Davis	B501	(777) 999-8888	Professor	501	Business School

5 rows in set (0.001 sec)

Offers table contents

```
MariaDB [university]> SELECT * FROM Offers;
```

Course_number	DepartmentCode
BUS501	501
CS101	101
EE301	301
ENGL401	401
MATH201	201

5 rows in set (0.001 sec)

Section table contents

```
MariaDB [university]> SELECT * FROM Section;
```

Section_number	Semester	Year	Timing	Classroom	Building	FacultyID	Course_number
10101	Fall	2024	MWF 10:00 AM - 11:00 AM	Room 101	Science Building	1001	CS101
20101	Spring	2025	TTH 1:00 PM - 2:30 PM	Room 201	Main Building	1002	MATH201
30101	Fall	2024	MW 3:00 PM - 4:30 PM	Room 301	Engineering Building	1003	EE301
40101	Fall	2024	TTH 9:00 AM - 10:30 AM	Room 401	Arts Building	1004	ENGL401
50101	Spring	2025	MWF 11:00 AM - 12:00 PM	Room 501	Business Building	1005	BUS501

5 rows in set (0.001 sec)

Studies_in table contents

```
MariaDB [university]> SELECT * FROM Studies_in;
```

StudentID	Section_number
10001	10101
20001	20101
30001	30101
40001	40101
50001	50101

5 rows in set (0.001 sec)

Student table contents

```
MariaDB [university]> SELECT * FROM Student;
```

StudentID	Name	Address	Nationality	DateOfBirth	Major	DepartmentCode	CollegeName
10001	Sarah Johnson	123 Main St	American	1998-05-10	Computer Science	101	University of Science
20001	Daniel Wilson	456 Elm St	Canadian	1999-08-15	Mathematics	201	City College
30001	Jessica Lee	789 Oak St	British	2000-02-20	Electrical Engineering	301	Tech Institute
40001	Ryan Martinez	321 Pine St	Spanish	1997-11-30	English Literature	401	Liberal Arts College
50001	Emma Taylor	654 Cedar St	Australian	1996-04-25	Business Administration	501	Business School

5 rows in set (0.002 sec)

Works_for table contents

```
MariaDB [university]> SELECT * FROM Works_for;
```

FacultyID	DepartmentCode
1001	101
1002	201
1003	301
1004	401
1005	501

5 rows in set (0.001 sec)

- Python code:

```
import mysql.connector

# Function to connect to the database
def connect_to_database():
    try:
        connection = mysql.connector.connect(
            host="localhost",
            user="HIBA",
            password="102938AFhn",
            database="university"
        )
        return connection
    except mysql.connector.Error as error:
        print("Error connecting to database:", error)
        return None

# Function to fetch and display all courses
def fetch_courses(connection):
    try:
        cursor = connection.cursor()
        cursor.execute("SELECT * FROM Course")
        rows = cursor.fetchall()
        print("\nCourses:")
        for row in rows:
            print(row)
    except mysql.connector.Error as error:
        print("Failed to fetch courses:", error)

# Function to fetch and display all departments
def fetch_departments(connection):
    try:
        cursor = connection.cursor()
        cursor.execute("SELECT * FROM Department")
        rows = cursor.fetchall()
        print("\nDepartments:")
        for row in rows:
            print(row)
```

```

except mysql.connector.Error as error:
    print("Failed to fetch departments:", error)

# Function to enroll a student in a course
def enroll_in_course(connection, student_id, section_number):
    try:
        cursor = connection.cursor()
        cursor.execute("INSERT INTO Studies_in (StudentID, Section_number) VALUES (%s, %s)", (student_id,
section_number))
        connection.commit()
        print("Enrolled in course successfully!")
    except mysql.connector.Error as error:
        print("Failed to enroll in course:", error)

# Function to fetch and display teaching schedule for a faculty member
def fetch_teaching_schedule(connection, faculty_id):
    try:
        cursor = connection.cursor()
        cursor.execute("SELECT * FROM Section WHERE FacultyID = %s", (faculty_id,))
        rows = cursor.fetchall()
        print("\nTeaching Schedule:")
        for row in rows:
            print(row)
    except mysql.connector.Error as error:
        print("Failed to fetch teaching schedule:", error)

# Function to view students enrolled in a specific section
def view_enrolled_students(connection, faculty_id, section_number):
    try:
        cursor = connection.cursor()
        cursor.execute("SELECT s.StudentID, s.Name FROM Studies_in si JOIN Student s ON si.StudentID =
s.StudentID WHERE si.Section_number = %s", (section_number,))
        rows = cursor.fetchall()
        print("\nEnrolled Students:")
        for row in rows:
            print(row)
    except mysql.connector.Error as error:
        print("Failed to fetch enrolled students:", error)

```

```
# Function to fetch and display student information
```

```
def fetch_student_info(connection, student_id):
```

```
    try:
```

```
        cursor = connection.cursor()
```

```
        cursor.execute("SELECT * FROM Student WHERE StudentID = %s", (student_id,))
```

```
        row = cursor.fetchone()
```

```
        print("\nStudent Information:")
```

```
        print(row)
```

```
    except mysql.connector.Error as error:
```

```
        print("Failed to fetch student information:", error)
```

```
# Function to search for courses by name
```

```
def search_courses(connection, course_name):
```

```
    try:
```

```
        cursor = connection.cursor()
```

```
        cursor.execute("SELECT * FROM Course WHERE Course_Name LIKE %s", ("%"+course_name+"%",))
```

```
        rows = cursor.fetchall()
```

```
        print("\nSearch Results:")
```

```
        for row in rows:
```

```
            print(row)
```

```
    except mysql.connector.Error as error:
```

```
        print("Failed to search courses:", error)
```

```
# Function to update student details
```

```
def update_student_details(connection, student_id, new_major):
```

```
    try:
```

```
        cursor = connection.cursor()
```

```
        cursor.execute("UPDATE Student SET Major = %s WHERE StudentID = %s", (new_major, student_id))
```

```
        connection.commit()
```

```
        print("Student details updated successfully!")
```

```
    except mysql.connector.Error as error:
```

```
        print("Failed to update student details:", error)
```

```
# Function to fetch and display all sections
```

```
def fetch_sections(connection):
```

```
    try:
```

```
        cursor = connection.cursor()
```

```
        cursor.execute("SELECT * FROM Section")
```

```
        rows = cursor.fetchall()
```



```

    print("\nSections:")
    for row in rows:
        print(row)
except mysql.connector.Error as error:
    print("Failed to fetch sections:", error)

# Main function to demonstrate database interaction
def main():
    # Connect to the database
    connection = connect_to_database()
    if connection:
        while True:
            print("\nOptions:")
            print("1. View Courses")
            print("2. View Departments")
            print("3. Enroll in Course")
            print("4. Faculty Operations")
            print("5. View Student Information")
            print("6. Search Courses")
            print("7. Update Student Details")
            print("8. View Sections")
            print("9. Exit")
            choice = input("Enter your choice (1-9): ")

            if choice == "1":
                fetch_courses(connection)
            elif choice == "2":
                fetch_departments(connection)
            elif choice == "3":
                student_id = input("Enter your student ID: ")
                section_number = input("Enter section number to enroll in: ")
                enroll_in_course(connection, student_id, section_number)
            elif choice == "4":
                faculty_id = input("Enter faculty ID: ")
                print("\nFaculty Operations:")
                print("1. View Teaching Schedule")
                print("2. View Enrolled Students")
                faculty_choice = input("Enter your choice (1-2): ")
                if faculty_choice == "1":

```

```

        fetch_teaching_schedule(connection, faculty_id)
    elif faculty_choice == "2":
        section_number = input("Enter section number: ")
        view_enrolled_students(connection, faculty_id, section_number)
    else:
        print("Invalid choice!")
    elif choice == "5":
        student_id = input("Enter student ID: ")
        fetch_student_info(connection, student_id)
    elif choice == "6":
        course_name = input("Enter course name to search: ")
        search_courses(connection, course_name)
    elif choice == "7":
        student_id = input("Enter student ID: ")
        new_major = input("Enter new major: ")
        update_student_details(connection, student_id, new_major)
    elif choice == "8":
        fetch_sections(connection)
    elif choice == "9":
        print("Exiting program.")
        break
    else:
        print("Invalid choice!")

# Close the connection
connection.close()
print("Connection closed.")

if __name__ == "__main__":
    main()

```

- Outputs:

Option 1: View Courses

```
Options:
1. View Courses
2. View Departments
3. Enroll in Course
4. Faculty Operations
5. View Student Information
6. Search Courses
7. Update Student Details
8. View Sections
9. Exit
Enter your choice (1-9): 1

Courses:
('BUS501', 'Principles of Marketing', 'Introduction to marketing principles and strategies.', 3, 501)
('CS101', 'Introduction to Computer Science', 'Fundamental concepts of computer science.', 3, 101)
('EE301', 'Introduction to Electrical Engineering', 'Fundamental concepts of electrical engineering.', 3, 301)
('ENGL401', 'Shakespearean Literature', 'Study of Shakespeare's plays and sonnets.', 3, 401)
('MATH201', 'Calculus I', 'Fundamental concepts of calculus.', 4, 201)
```

Option 2: View Departments

```
Options:
1. View Courses
2. View Departments
3. Enroll in Course
4. Faculty Operations
5. View Student Information
6. Search Courses
7. Update Student Details
8. View Sections
9. Exit
Enter your choice (1-9): 2

Departments:
(101, 'Computer Science', 'CS101', 'University of Science')
(201, 'Mathematics', 'MATH201', 'City College')
(301, 'Electrical Engineering', 'EE301', 'Tech Institute')
(401, 'English Literature', 'ENGL401', 'Liberal Arts College')
(501, 'Business Administration', 'BUS501', 'Business School')
```

Option 3: Enroll in Course

```
Options:
1. View Courses
2. View Departments
3. Enroll in Course
4. Faculty Operations
5. View Student Information
6. Search Courses
7. Update Student Details
8. View Sections
9. Exit
Enter your choice (1-9): 3
Enter your student ID: 10001
Enter section number to enroll in: 20101
Enrolled in course successfully!
```

Option 4: Faculty Operations (View Teaching Schedule)

```
Options:
1. View Courses
2. View Departments
3. Enroll in Course
4. Faculty Operations
5. View Student Information
6. Search Courses
7. Update Student Details
8. View Sections
9. Exit
Enter your choice (1-9): 4
Enter faculty ID: 1001

Faculty Operations:
1. View Teaching Schedule
2. View Enrolled Students
Enter your choice (1-2): 1

Teaching Schedule:
(10101, 'Fall', 2024, 'MWF 10:00 AM - 11:00 AM', 'Room 101', 'Science Building', 1001, 'CS101')
```

Option 4: Faculty Operations (View Enrolled Students)

```
Options:
1. View Courses
2. View Departments
3. Enroll in Course
4. Faculty Operations
5. View Student Information
6. Search Courses
7. Update Student Details
8. View Sections
9. Exit
Enter your choice (1-9): 4
Enter faculty ID: 1002

Faculty Operations:
1. View Teaching Schedule
2. View Enrolled Students
Enter your choice (1-2): 2
Enter section number: 20101

Enrolled Students:
(10001, 'Sarah Johnson')
(20001, 'Daniel Wilson')
```

Option 5: View Student Information

```
Options:
1. View Courses
2. View Departments
3. Enroll in Course
4. Faculty Operations
5. View Student Information
6. Search Courses
7. Update Student Details
8. View Sections
9. Exit
Enter your choice (1-9): 5
Enter student ID: 10001

Student Information:
(10001, 'Sarah Johnson', '123 Main St', 'American', datetime.date(1998, 5, 10), 'computer science', 101, 'University of Science')
```

Option 6: Search Courses

```
Options:
1. View Courses
2. View Departments
3. Enroll in Course
4. Faculty Operations
5. View Student Information
6. Search Courses
7. Update Student Details
8. View Sections
9. Exit
Enter your choice (1-9): 6
Enter course name to search: calculus I

Search Results:
('MATH201', 'Calculus I', 'Fundamental concepts of calculus.', 4, 201)
```

Option 8: View Sections

```
Options:
1. View Courses
2. View Departments
3. Enroll in Course
4. Faculty Operations
5. View Student Information
6. Search Courses
7. Update Student Details
8. View Sections
9. Exit
Enter your choice (1-9): 8

Sections:
(10101, 'Fall', 2024, 'MWF 10:00 AM - 11:00 AM', 'Room 101', 'Science Building', 1001, 'CS101')
(20101, 'Spring', 2025, 'TTH 1:00 PM - 2:30 PM', 'Room 201', 'Main Building', 1002, 'MATH201')
(30101, 'Fall', 2024, 'MW 3:00 PM - 4:30 PM', 'Room 301', 'Engineering Building', 1003, 'EE301')
(40101, 'Fall', 2024, 'TTH 9:00 AM - 10:30 AM', 'Room 401', 'Arts Building', 1004, 'ENGL401')
(50101, 'Spring', 2025, 'MWF 11:00 AM - 12:00 PM', 'Room 501', 'Business Building', 1005, 'BUS501')
```

Option 7: Update Student Information

```
Options:
1. View Courses
2. View Departments
3. Enroll in Course
4. Faculty Operations
5. View Student Information
6. Search Courses
7. Update Student Details
8. View Sections
9. Exit
Enter your choice (1-9): 7
Enter student ID: 10001
Enter new major: BUSINESS
Student details updated successfully!
```

Option 9: Exit

```
Options:
1. View Courses
2. View Departments
3. Enroll in Course
4. Faculty Operations
5. View Student Information
6. Search Courses
7. Update Student Details
8. View Sections
9. Exit
Enter your choice (1-9): 9
Exiting program.
Connection closed.
```

Experiences/

- Working on the Student Records System was an amazing learning experience for us.
- It was tough to design the database, but we learned a lot about dealing with complicated connections between data and making sure the information stayed accurate via MySQL and localhost.
- Building the frontend was also challenging. We had the opportunity to use python to create user-friendly pages. Connecting these pages to the backend taught us how important it is to handle data smoothly in order to avoid errors.
- Debugging was hard, but it was crucial. We became better at finding and fixing problems.
- Working as a team was extremely important. We had to communicate clearly and regularly in order to stay on track.
- Overall, we improved our technical skills, problem-solving abilities, and teamwork. This project definitely prepared us for future challenges.

References/

- **Elmasri, R. and Navathe, S. (2020) Fundamentals of Database Systems. Hoboken, NJ: Pearson.**